

仿生人类四层记忆模型 AI 记忆系统设计方案

仿生人类四层记忆模型 AI 记忆系统设计方案

文档目录

- 项目概述
- 理论依据（四大人类记忆心理学模型）
- 环境依赖配置
- 分层代码实现（瞬时记忆 / 工作记忆 / 短时向量记忆 / 长时图谱记忆）
- 顶层调度 Agent 代码
- 程序调用示例
- 模块与心理学模型对应说明
- 后续优化拓展方案

一、项目概述

本方案依托阿特金森多存储模型、Baddeley 工作记忆模型、LOP 加工水平模型、海马 + 联结主义记忆模型四大经典认知理论，搭建四层分级 AI 记忆架构，解决大模型上下文超限失忆、关键信息被闲聊覆盖、跨会话记忆丢失问题。

整体四层流转链路：瞬时感官记忆→Baddeley 工作记忆→短时向量记忆库→长时知识图谱固化记忆。

二、理论对应关系

人类记忆模型	实现模块	核心作用
Atkinson-Shiffrin 多存储	瞬时 + 短时 + 长时三级存	信息分级流转，超出上下

模型	储架构	文不直接销毁
Baddeley 工作记忆模型	工作记忆四大组件：中央执行器、文本环路、视空画板、情景缓冲器	实时对话信息管控、多模态融合
LOP 加工水平理论	深浅信息分级编码	浅层闲聊丢弃，关键信息深度编码入库
海马巩固 + 联结主义	定时蒸馏任务 + Neo4j 知识图谱	模拟睡眠固化记忆，结构化永久存储

三、环境依赖配置

1、Python 安装依赖命令

Plain Text

```
pip install langchain langchain-openai faiss-cpu redis py2neo sentence-transformers
```

2、中间件部署要求

1. Redis：本地 127.0.0.1:6379，用于缓存瞬时、工作记忆；
2. FAISS：本地向量数据库，存储 7 天有效期短时记忆；
3. Neo4j：127.0.0.1:7687，账号 neo4j，自定义密码，存储长期联结记忆。

3、全局常量配置

```
python
# 导入依赖包
import redis
import faiss
import numpy as np
from langchain.vectorstores import FAISS
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.chat_models import ChatOpenAI
from py2neo import Graph, Node, Relationship
from datetime import datetime, timedelta
```

```

import threading

# 全局初始化
embed_model = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
r = redis.Redis(host="127.0.0.1", port=6379, db=0,
decode_responses=True)
short_index = FAISS.from_texts(["init"], embed_model)
graph = Graph("bolt://127.0.0.1:7687", auth=("neo4j", "123456"))

# 业务常量
SHORT_MEM_EXPIRE = timedelta(days=7)
DISTILL_CYCLE = 86400

```

四、分层代码实现

4.1 第一层：瞬时感官记忆（对标感觉记忆）

```

python
class InstantMemory:
    def __init__(self, session_id):
        self.key = f"instant:{session_id}"
        self.max_token = 8000

    def add_msg(self, role, content):
        msgs = r.lrange(self.key, 0, -1)
        msgs.append(f"{role}:{content}")
        if len("\n".join(msgs)) > self.max_token*0.9:
            msgs = msgs[-int(len(msgs)*0.6):]
        r.delete(self.key)
        r.rpush(self.key, *msgs)
        return "\n".join(msgs)

    def clean_useless(self):
        raw = "\n".join(r.lrange(self.key,0,-1))
        prompt = f"""区分下面对话内容：
1.浅层信息(闲聊、无价值废话):标记【LOW】
2.深层信息(用户设定、需求、关键数据、规则):标记【HIGH】
原文:{raw}
只输出标记+分段内容"""
        res = llm.invoke(prompt).content
        low_text, high_text = self.split_level(res)

```

```

        r.delete(self.key)
        return high_text

    def split_level(self, txt):
        low, high = [], []
        for seg in txt.split("【"):
            if "LOW】" in seg:
                low.append(seg.split("】")[1])
            elif "HIGH】" in seg:
                high.append(seg.split("】")[1])
        return "\n".join(low), "\n".join(high)

```

4.2 第二层：Baddeley 工作记忆

```

python
class BaddeleyWorkingMem:
    def __init__(self, session_id):
        self.sid = session_id
        self.executor = llm
        self.text_loop_key = f"work:text:{session_id}"
        self.img_space_key = f"work:img:{session_id}"
        self.buffer_key = f"work:buffer:{session_id}"

    def add_text(self, text):
        r.rpush(self.text_loop_key, text)
        if r.llen(self.text_loop_key) > 15:
            r.lpop(self.text_loop_key)

    def add_multimodal(self, img_desc):
        r.rpush(self.img_space_key, img_desc)

    def scene_fusion(self):
        txt_data = "\n".join(r.lrange(self.text_loop_key, 0, -1))
        img_data = "\n".join(r.lrange(self.img_space_key, 0, -1))
        fusion_prompt = f"整合对话和图像信息，精简为结构化摘要：对话{txt_data}，图像信息{img_data}"
        scene_summary =
self.executor.invoke(fusion_prompt).content
        r.set(self.buffer_key, scene_summary)
        return scene_summary

```

4.3 第三层：短时 FAISS 向量记忆

```
python
class ShortTermMem:
    def __init__(self):
        self.vector_store = short_index
        self.save_time = {}

    def save_memory(self, content, save_time=datetime.now()):
        self.vector_store.add_texts([content])
        self.save_time[len(self.save_time)] = save_time

    def recall_memory(self, query, top_k=3):
        docs = self.vector_store.similarity_search(query, k=top_k)
        return "\n".join([d.page_content for d in docs])

    def auto_expire_clean(self):
        now = datetime.now()
        del_list = []
        for idx,t in self.save_time.items():
            if now-t > SHORT_MEM_EXPIRE:
                del_list.append(idx)
        for d in del_list:
            del self.save_time[d]
```

4.4 第四层：长时海马蒸馏 + 知识图谱记忆

```
python
class LongTermMem:
    def __init__(self, short_mem:ShortTermMem):
        self.short_mem = short_mem
        threading.Thread(target=self.hippocampus_distill_loop,
        daemon=True).start()

    def hippocampus_distill_loop(self):
        import time,json
        while True:
            time.sleep(DISTILL_CYCLE)
            all_short_data = [doc.page_content for doc in
self.short_mem.vector_store.similarity_search("",k=999)]
            if not all_short_data:
                continue
            distill_prompt = f"从文本提取【实体，属性，关系】，标准
JSON 数组输出: {all_short_data}"
            distill_res = llm.invoke(distill_prompt).content
```

```

        try:
            data = json.loads(distill_res)
            for item in data:
                n1 = Node(item["type"], name=item["entity1"])
                n2 = Node(item["type2"], name=item["entity2"])
                rel = Relationship(n1, item["relation"], n2)
                graph.create(n1|n2|rel)
        except Exception as e:
            pass

    def kg_recall(self, query):
        cypher = f"""MATCH (n)-[r]->(m) WHERE n.name CONTAINS
'{query}' OR m.name CONTAINS '{query}' RETURN
n.name,type(r),m.name"""
        res = graph.run(cypher).data()
        return str(res)

```

五、顶层总调度 Agent 代码

```

python
class HumanLikeMemoryAgent:
    def __init__(self, session_id):
        self.sid = session_id
        self.instant = InstantMemory(session_id)
        self.working = BaddeleyWorkingMem(session_id)
        self.short = ShortTermMem()
        self.long = LongTermMem(self.short)

    def chat_step(self, user_input, img_info=None):
        self.instant.add_msg("user", user_input)
        if img_info:
            self.working.add_multimodal(img_info)
        high_info = self.instant.clean_useless()
        if high_info:
            self.working.add_text(high_info)
            scene = self.working.scene_fusion()
            self.short.save_memory(scene)
        recall_short = self.short.recall_memory(user_input)
        recall_long = self.long.kg_recall(user_input)
        final_prompt = f"历史记录:{recall_short}\n 长期存
档:{recall_long}\n 用户问题:{user_input}"
        ans = llm.invoke(final_prompt).content
        self.instant.add_msg("assistant", ans)

```

```
return ans
```

六、调用示例

```
python
# 创建会话
agent = HumanLikeMemoryAgent(session_id="user_001")
# 多轮对话
print(agent.chat_step("我的名字叫张三，今年 25 岁，职业程序员"))
print(agent.chat_step("帮我规划一个学习路线"))
# 新建同用户会话，跨周期读取历史记忆
agent2 = HumanLikeMemoryAgent(session_id="user_001")
print(agent2.chat_step("我是谁? "))
```

七、模块与心理学模型对应说明

1. **多存储模型**：瞬时→工作→短时→长时四级流转，复刻感觉记忆、短时记忆、长时记忆三级存储逻辑；
2. **Baddeley 模型**：工作记忆类四分结构，中央执行器负责调度，文本 / 视空缓存对应双通道，情景缓冲作为中转；
3. **LOP 深度加工**：瞬时层自动区分深浅内容，浅信息丢弃、深信息入库，编码深度决定存储生命周期；
4. **海马 + 联结主义**：每日定时蒸馏模拟睡眠巩固记忆，知识图谱模拟人脑神经元联结网络，永久存储。

八、优化拓展方案

1. **回忆强化机制**：每次调取长时记忆后重新摘要入库，模拟人脑反复回忆加固记忆；
2. **动态遗忘权重**：长期未调用的记忆向量衰减权重，复刻艾宾浩斯遗忘曲线；
3. **多用户隔离**：Redis、向量库按用户 ID 分片，多用户记忆互不干扰。